# Embedded Systems Project Planning

https://gemini.google.com/app/f28bf499147f03bb

*User prompt: there is a capacitor connecting vcc1 and vcc2 of the hbridge to 5v and 12v*

Response: If you have capacitors bridging directly *between* VCC1 and VCC2, you should remove them immediately.

While decoupling capacitors are excellent for stabilizing power, they must be wired from VCC1 to Ground and VCC2 to Ground individually. Bridging VCC1 and VCC2 together with a capacitor can couple high-voltage 12V motor transients directly into the 5V logic supply, which can cause erratic behavior or damage the SN754410NE.

Also, remember the strict hardware rule for Checkpoint 1: both the 5V logic supply (VCC1) and the 12V motor supply (VCC2) must come from the in-class DC power supply, never from the RP4.

While properly wired decoupling capacitors will help clean up high-frequency switching noise from the motor, they won't fix the 60Hz sine wave you are seeing on the enable pin. That 60Hz AC noise confirms the enable pin is floating. Adding a 10kΩ pull-down resistor from the H-bridge enable pins to ground will ensure they are actively pulled low whenever the 4N25 optocoupler turns off, fulfilling the requirement that the motor defaults to a disabled state.

---

*User prompt: the ground was connected and now there is no sine wave. I am also seeing square wave inputs into all of the hbridge inputs. however no output is moving the motor*

Response: That is excellent news. Seeing clean square waves on all four H-bridge inputs means your C controller, ARM assembly timing loop, and the Raspberry Pi GPIOs are working perfectly. The software side is solid.

Since the signals are reaching the SN754410NE but the motor isn't moving, the bottleneck is entirely on the power or output stage.

Check these specific hardware points:

- **Verify the Enable Voltage (Pins 1 and 9):** While the RP4 is outputting the enable signal, the 4N25 optocoupler must physically pass 5V to the SN754410NE enable pins to turn the outputs on. Press E in your program to enable the motor, and measure pins 1 and 9 on the H-bridge. If they are sitting at 0V instead of ~5V, your optocoupler circuit is not pulling the pins high.
- **Check the DC Supply Current Limit:** The Nema17 stepper draws about 0.35 A per phase. Check the in-class DC power supply screen. When you press E to enable the motor, the VCC2 current should

clearly rise to roughly 0.5 A to 0.7 A. If the supply's current limit is set too low, it will choke the voltage the moment the coils try to draw power.

- **Probe VCC1 and VCC2 at the Chip:** Measure directly on the SN754410NE legs. Pin 16 (VCC1) must be 5.0 V and Pin 8 (VCC2) must be 12.0 V. If either is missing, the internal logic or the motor drive will fail to operate.
- **Probe the Output Pins (1Y, 2Y, 3Y, 4Y):** Check the actual output pins going to the motor. If you see 12V square waves here but the motor isn't moving, the motor coils are wired to the wrong pairs (e.g., mixing a Coil A wire with a Coil B wire). Coil 1 must be strictly on 1Y/2Y, and Coil 2 must be strictly on 3Y/4Y.

---

*User prompt: i got the motor spinning however, there are a few bugs that need to get worked out*

Response: Getting a stepper motor to spin for the first time is the biggest hurdle of the hardware phase, so excellent work crossing that line.

What specific bugs are you seeing now? Are you experiencing missed steps or erratic buzzing at higher RPMs, is the speed scaling behaving unexpectedly when you press 'U' or 'D', or are you running into issues executing the continuous run and stop commands?

---

*User prompt: before we start, here is all of the updated code*

Response: I have reviewed the updated main.c, motor_driver.s, and Makefile. The additions of the T test command for individual step progression and the get_timer_frequency assembly routine are integrated correctly to help isolate timing and logic issues.

Describe the specific bugs you are observing now that the motor is spinning. Are you experiencing skipped steps at higher RPMs, erratic behavior when changing directions, or issues with the U and D speed scaling commands?

---

*User prompt: when the program is first started and before the motor is enabled, the motor turns freely like it is supposed to. after enabling the motor, it still turns freely, which it is not supposed to do. after starting the motor, it starts to spin, correct. after stopping, the motor does not spin freely, correct. after disabling the motor, the motor cannot move freely, incorrect. the motor wont turn freely until after the program is stopped.*

Response: The symptoms you describe map perfectly to two separate issues: one in the software's initial state, and one in the hardware's TTL logic.

**1. The Disable Bug (Hardware: Floating TTL Inputs)** When you press E to disable the motor, your C program correctly sets the RP4 GPIO to 0, turning off the 4N25 optocoupler. However, the SN754410NE is a TTL logic chip. When the 4N25 switch opens, the SN754410NE enable pins (1 and 9) are left disconnected, causing them to "float" to a logic HIGH state. Because they float HIGH, the H-bridge remains fully enabled and continues to drive the last known coil pattern to the motor, keeping it locked.

The only reason quitting the program (Q) frees the motor is because the cleanup routine forces the actual

motor coils (pins A, B, C, and D) to 0V, dropping the holding torque.

- **The Hardware Fix:** To fulfill the project requirement that the motor is disabled whenever the optocoupler is off or floating, you must wire a pull-down resistor (e.g., 10kΩ) between the SN754410NE enable pins and ground. This actively pulls the voltage to 0V when the 4N25 turns off.

**2. The Enable Bug (Software: Zeroed Coil State)** When you first start the program, the test_motor_gpio() function leaves all four motor pins (A, B, C, D) at 0V. When you press E to enable the motor, the H-bridge turns on, but it applies 0V to all coils. A stepper motor only holds its position when a specific coil pattern (like 0b1010) is applied. The motor turns freely because the coils are completely de-energized until you press S, which is the first time step_routine actually sends a holding pattern to the pins.

- **The Software Fix:** We need to update the E case in your main.c switch statement to immediately apply the current step pattern when enabled. We can also add a software failsafe to forcefully de-energize the coils when disabled, mimicking what the Q command does.

Update the case 'e': block inside your main.c while loop to match this:

```
                case 'e':
                case 'E': // Enable/Disable H-Bridge
                    enabled = !enabled;
                    enable_routine(PIN_ENABLE, enabled);

                    if (enabled) {
                        // Software Fix: Apply the current coil pattern
immediately
                        // so the motor snaps into holding torque right when
enabled.
                        step_routine(direction, mode, motor_pins);
                    } else {
                        // Software Failsafe: De-energize all coils. This
forces the
                        // motor to free up even if the hardware enable pins
are floating[cite: 8].
                        for (int i = 0; i < 4; i++) {
                            E4235_Write(motor_pins[i], 0); //[cite: 46, 57]
                        }
                    }

                    printf("Motor %s.\n", enabled ? "ENABLED" : "DISABLED");
                    break;
```

Compile and run this change. The motor will immediately lock when you press E initially, and the software failsafe will forcefully drop the holding torque when you disable it. Ensure your lab partner still adds the pull-down resistor to the circuit so the schematic passes the Checkpoint 1 hardware safety requirements.

---

*User prompt: i think all functions are now working*

Response: With the motor spinning and all keyboard commands functioning, the final hardware requirement to complete Checkpoint 1 is the stall-speed characterization.

To run this characterization using your C controller:

1. Ensure the propeller is securely fitted to the motor shaft.
2. Press E to enable and S to start the motor at a low initial step rate.
3. Press U repeatedly to increase the speed by 5% increments until the motor stalls, which will look like buzzing, erratic motion, or stopping entirely.
4. Note the final steps/second printed in your terminal just before the stall.
5. Convert that rate to RPM: RPM = steps per second * 60 / steps per revolution (200 for full-step, 400 for half-step).
6. Repeat this process for three separate runs to find your average stall speed, which must be compared against the 100 RPM limit.

Once your lab partner finishes collecting those numbers, would you like to begin drafting the human-written Checkpoint 1 Strategy Document or the Pre-Power Inspection PDF?

---

*User prompt: what else is left for checkpoint 1*

Response: For Checkpoint 1, your team still needs to complete and submit the required documentation and organize your repository.

Here are the remaining deliverables:

- **Strategy Document (Human-written PDF):** This document must detail your team members, phone platform, required resources, AI tools, AI-documentation plan, app plan, work division, and schedule.
- **Pre-Power Inspection (Human-written):** This must be documented as completed before power was first applied. It needs to verify that VCC1 and VCC2 come from the DC power supply rather than the RP4, check the common ground, SN754410NE pinout, 4N25 enable wiring, logic levels, and current/flyback protection, and record any problems found.
- **AI-Generated Hardware Documentation:** You must submit a preliminary motor schematic, assembly instructions, and a parts list. This package must be stored in the CP1 repository and submitted on ELC.
- **Submission and Repository Organization:** The human-written documents must be submitted on ELC as both a PDF and an editable version. Your CP1 repository must also be organized and clearly labeled.

Do you want to start outlining the Strategy Document, or should we prompt the AI to generate the preliminary schematic, assembly instructions, and parts list based on your lab partner's final wiring?

---

*User prompt: can you make it display the step mode of the motor. like if it is running at full or half steops*

Response: The Checkpoint 1 specifications require the step mode (full or half) to be printed at start-up, and the current step rate to be printed after every speed change. While the current code already prints the mode at initialization, you can append it to your continuous run and speed change outputs so it remains explicitly visible while the motor is operating.

Update the S, U, and D cases in your main.c switch statement to include the mode string:

```
                case 's':
                case 'S': // Continuous Run / Stop
                    running = !running;
                    printf("Motor %s in %s mode.\n", running ? "RUNNING" :
"STOPPED", mode == 0 ? "Full Step" : "Half Step");
                    break;

                case 'u':
                case 'U': // Faster (increase by ~5%)
                    steps_per_sec *= 1.05; //[cite: 7]
                    printf("Speed increased. Current rate: %.2f steps/sec
(%s)\n", steps_per_sec, mode == 0 ? "Full Step" : "Half Step");
                    break;

                case 'd':
                case 'D': // Slower (decrease by ~5%)
                    steps_per_sec *= 0.95; //[cite: 7]
                    if (steps_per_sec < 10.0) steps_per_sec = 10.0; // Prevent
stalling at zero
                    printf("Speed decreased. Current rate: %.2f steps/sec
(%s)\n", steps_per_sec, mode == 0 ? "Full Step" : "Half Step");
                    break;
```

---

*User prompt: when using step and t to advance to the next step, the motor is disabled and does not move*

Response: The motor is not moving because the code for the T command explicitly disables the H-bridge before taking a step. In your current main.c, the T case sets enabled = 0; and calls enable_routine(PIN_ENABLE, 0);.

While this is useful for safely measuring the logic signals on the H-bridge inputs with a multimeter without drawing current, it prevents the SN754410NE from energizing the motor coils to physically move the shaft.

To make the motor physically advance one step when you press T, the H-bridge must remain enabled. Update the T case in your switch statement to ensure the motor is enabled before stepping:

```
                case 't':
                case 'T': // Test one assembly motor step
                    running = 0; // Stop continuous run

                    // To physically move the motor, the H-bridge must be
enabled
                    if (!enabled) {
```

```
                        enabled = 1;
                        enable_routine(PIN_ENABLE, 1);
                        printf("Motor automatically ENABLED for step.\n");
                    }

                    // Advance the assembly stepping sequence once
                    step_routine(direction, mode, motor_pins);

                    printf("\nAssembly step executed.\n");
                    printf("Direction: %s\n", direction ? "Clockwise" :
"Counterclockwise");
                    printf("Motor stepped. Measure SN754410NE input pins 2, 7,
10, and 15 if needed.\n");
                    printf("Press T to advance to the next step.\n\n");
                    break;
```

---

*User prompt: could you make it so that it does not affect the motor enable or disable control? t will only move the motor when the motor is already enabled. if the motor is disabled, it will not move the motor*

Response: Here is the updated T case. It now mirrors the logic of your spacebar command by simply checking the enabled flag first. If the motor is disabled, it will halt the continuous run but refuse to step the motor, leaving your enable/disable state completely untouched.

Replace the case 't': block in your switch statement with this:

```
                case 't':
                case 'T': // Test one assembly motor step
                    running = 0; // Stop continuous run

                    if (enabled) {
                        // Advance the assembly stepping sequence once
                        step_routine(direction, mode, motor_pins);

                        printf("\nAssembly step executed.\n");
                        printf("Direction: %s\n", direction ? "Clockwise" :
"Counterclockwise");
                        printf("Motor stepped. Measure SN754410NE input pins 2,
7, 10, and 15 if needed.\n");
                        printf("Press T to advance to the next step.\n\n");
                    } else {
                        printf("\nCannot step: Motor is currently DISABLED and
will not move.\n");
                        printf("Press E to enable the motor first.\n\n");
                    }
                    break;
```

---

*User prompt: i changed the code to use half steps for testing purposed. the output of xxx steps per second*

*for a half step, is that half steps per second? or full steps?*

Response: It refers to **half steps per second**.

The steps_per_sec variable dictates exactly how many times your C program calls the assembly step_routine each second. Because you switched to half-step mode, every single call to that routine now advances the sequence by just one half-step (cycling through the 8-step table instead of the 4-step table).

If your terminal prints "100 steps/sec" while in half-step mode, the motor is executing 100 half-steps every second. Since a standard Nema17 requires 400 half-steps to complete a single 360-degree revolution, running at 100 steps/sec in this mode means it will take exactly 4 seconds to turn one full rotation (15 RPM).

---

*User prompt: we need to make some tweaks to the code. currently, we are only acheiving about 100 rpm. is there a way to bring up the speed of the motor before it begins to stall*

Response: Change the initial mode variable in main.c back to 0 (Full Step). Half-stepping provides smoother motion at low speeds but reduces overall torque because only one coil is energized during the intermediate steps. Full-stepping keeps both coils fully energized at all times, providing the maximum possible magnetic holding force to push through higher RPMs before stalling.

Ensure you are ramping up the speed gradually. Stepper motors cannot instantly jump to a high speed from a dead stop due to rotor inertia. Start the motor at a low speed (e.g., 50 steps/sec) and wait for it to stabilize. Then, press U slowly, allowing the motor to fully accelerate to the new speed before pressing U again. If you increase the step rate faster than the rotor can physically accelerate, it will instantly stall and buzz.

Modify the U command to use a smaller multiplier for a gentler acceleration curve at high speeds. Changing the 5% increase to a 1% or 2% increase allows you to inch closer to the physical limit without overshooting it:

```
                case 'u':
                case 'U':
                    steps_per_sec *= 1.02; // Change to 2% increase for finer
high-speed control
                    printf("Speed increased. Current rate: %.2f steps/sec
(%s)\n", steps_per_sec, mode == 0 ? "Full Step" : "Half Step");
                    break;
```

Be aware that ~100 to 150 RPM is often the physical ceiling for a Nema 17 motor driven by a basic SN754410NE TTL H-bridge at 12V. Modern 3D printers achieve higher speeds using specialized "chopper" drivers (like the A4988) running at 24V, which aggressively force high voltage into the coils to overcome electromagnetic inductance. Because your project is restricted to a 12V DC supply and a simple H-bridge, the coil inductance physically prevents the current from rising fast enough during very short step pulses, starving the motor of torque at higher speeds.

---